

Chapter 3.

Functions

Nội dung

1. Giới thiệu chương trình con
2. Thiết kế chương trình con
3. Giới thiệu hàm đệ quy
4. Ví dụ

1.KHÁI NIỆM CHƯƠNG TRÌNH CON

- Để thuận tiện cho việc viết chương trình người ta thường tách chương trình đó thành các đoạn chương trình nhỏ; mỗi đoạn chương trình nhỏ này giải quyết *trọn vẹn* một vấn đề/công đoạn cụ thể của bài toán; mỗi đoạn chương trình nhỏ đó được gọi là một **chương trình con**.
- Chương trình con trong C/C++ được gọi chung là **hàm (function)**
 - Bản thân ngôn ngữ lập trình C/C++ đã được hỗ trợ một số hàm; các hàm này được gọi là các hàm chuẩn.
 - Tùy theo nhu cầu thực tế mà người lập trình có thể thiết kế thêm các hàm khác; các hàm này được gọi là các hàm tự tạo.

Lợi ích của việc sử dụng chương trình con

- Giúp thuận lợi trong việc kiểm thử, nâng cấp, viết tài liệu cho chương trình.
- Giúp rút ngắn thời gian lập trình.
- Giúp tránh lặp lại các đoạn lệnh tương tự nhau.
- Giúp cho việc phân tích thiết kế chương trình được thuận lợi,...

2. CÁCH THIẾT KẾ CHƯƠNG TRÌNH CON

- Các biến liệt kê trong các dòng *tiêu đề hàm* thường được gọi là các **tham số**; các tham số này gọi chung là tham số **hình thức**.
- Các tham số mà trước nó không có chỉ dẫn & thì được hiểu là tham số đó được truyền theo kiểu **tham trị**,
- Các tham số mà trước nó có chỉ dẫn & thì được hiểu là tham số đó được truyền theo kiểu **tham biến**.
- Danh sách các tham số ở *lời gọi hàm* được gọi là **tham số thực tế**.
- ❖ Hàm có thể không có tham số hoặc có nhiều tham số.

Phân biệt tham số hình thức trị/biến

- Các tham số được truyền theo kiểu tham trị thì những thay đổi trong thân hàm chứa nó sẽ không được giữ lại khi ra khỏi hàm chứa nó;
- Các tham số được truyền theo kiểu tham biến thì mọi sự thay đổi trong thân hàm chứa nó sẽ được giữ lại khi ra khỏi hàm chứa nó.
- ❖ Tham số được truyền theo kiểu tham biến là phương thức quan trọng để thiết kế hàm trả về nhiều giá trị.

Hãy cho biết kết quả của chương trình sau:

```
#include<iostream.h>
void test(int a, int &b, int &c); // tham so hình thức
int main()
{
    int x,y,z;
    x=1,y=2,z=3;
    test(x,y,z); // tham số thực tế
    cout<<x<<" "<<y<<" "<<z<<endl;
    return 0;
}
void test(int a, int &b, int &c)
{
    a=a+2; // a=3
    b=b+5; // b=7
    c=c+a+b; // c=3+3+7=13
}
```

Biến toàn cục và biến địa phương

- Các biến được khai báo trong các hàm - kể cả hàm `int main ()` được gọi là các **biến cục bộ** (biến địa phương) và nó chỉ có ảnh hưởng trong phạm vi thân hàm chứa nó. Khi hàm chứa nó kết thúc thì các biến này cũng mất tác dụng theo.
- Các biến được khai báo ngoài được gọi là các **biến toàn cục**; biến toàn cục ảnh hưởng đến cả chương trình.
- Khi lập trình, chúng ta cần tránh tối đa việc sử dụng các biến toàn cục.

Hãy cho biết kết quả của chương trình sau khi dữ liệu nhập vào là a=1 và b=2

```
#include<iostream.h>
void input(int &a, int &b);
void swap(int &a, int &b);
int main()
{
    int x,y;
    input(x,y);
    swap(x,y);
    cout<<x<<" "<<y<<endl;
    return 0;
}
void input(int &a, int &b)
{
    cout<<"Nhap 2 so a,b :";cin>>a>>b;
}
void swap(int &a, int &b)
{
    int temp=a;a=b;b=temp;
}
```

Ví dụ về việc
sử dụng hàm
không có
tham số;
biến toàn
cục, biến địa
phương

```
#include<iostream.h>
void nhap();
void tinhchuvi();
void tinhdientich();
int a,b; // biến toàn cục
int main()
{
    nhap();
    tinhchuvi();
    tinhdientich();
    cout<<endl;
    return 0;
}
void nhap()
{
    cout<<"Nhập a = ";cin>>a;
    cout<<"Nhập b = ";cin>>b;
}
void tinhchuvi()
{
    int cv=(a+b)*2; // biến địa phương
    cout<<"Chu vi HCN là : "<<cv<<endl;
}
void tinhdientich()
{
    int s=a*b; // biến địa phương
    cout<<"Diện tích HCN là : "<<s<<endl;
}
}
```

Hàm không có giá trị trả về, hàm có giá trị trả về

- Hàm không có giá trị trả về (trả về kiểu *void*), hàm có giá trị trả về (trả về kiểu khác *void*).
- Một hàm cần thiết kể có nhiều hơn một giá trị trả về thì kiểu trả về phải là *void*; và có thể trả về thông qua các tham biến.

Ví dụ:

Hàm nhập một phân số:

```
void nhapps(int &tu, int &mau)
```

Hàm tính tổng hai phân số, kết quả trả về là một phân số

```
void tongps(int tu1, int mau1, int tu2, int mau2, int &tu, int &mau)
```

- Một hàm cần thiết kế chỉ có một giá trị trả về thì có thể thiết kế hàm theo dạng hàm không có giá trị trả hoặc có giá trị trả về; khi đó ta cần thêm thông tin sau để quyết định việc lựa chọn một trong hai cách trên:
 - Nếu tên hàm không được sử dụng trong các hàm hoặc biểu thức khác nữa thì nên thiết kế theo dạng **hàm không có giá trị trả về**, ngược lại thì nên thiết kế theo dạng **hàm có giá trị trả về**.
 - ❖ Hàm được thiết kế theo dạng có giá trị trả về thì trong thân hàm đó phải có lệnh return để trả về giá trị là kết quả cần trả về của hàm đó.
- Lời gọi hàm:
 - Hàm có giá trị trả về được gọi như thế nào ?
 - Hàm không có giá trị trả về được gọi như thế nào ?

Ví dụ minh họa
cho lời gọi hàm
có giá trị trả
về/hàm không
có giá trị trả về

```
#include<iostream.h>
void input(int &a, int &b);
int HCN(int a, int b);
int main()
{
    int x,y;
    input(x,y);
    cout<<"Dien tich HCN la :"<<HCN(x,y);
    return 0;
}
void input(int &a, int &b)
{
    cout<<"Nhap a = "; cin>>a;
    cout<<"Nhap b = "; cin>>b;
}
int HCN(int a, int b)
{
    int s=a*b;
    return s;
}
```

Ví dụ:

Cho tam giác
ABC, biết tọa độ
3 điểm A,B,C.
Hãy tính diện
tích tam giác

```
#include<iostream.h>
#include<math.h>
void nhap(int &xa, int &ya, int &xb, int &yb, int &xc, int &yc);
float dientich(int xa, int ya, int xb, int yb, int xc, int yc);
int main()
{
    int xa,ya,xb,yb,xc,yc;
    nhap(xa,ya,xb,yb,xc,yc);
    cout<<"Dien tich cua tam giac : "<<dientich(xa,ya,xb,yb,xc,yc);
    cout<<endl;
    return 0;
}
void nhap(int &xa, int &ya, int &xb, int &yb, int &xc, int &yc)
{
    cout<<"Nhap toa do diem A : ";cin>>xa>>ya;
    cout<<"Nhap toa do diem B : ";cin>>xb>>yb;
    cout<<"Nhap toa do diem C : ";cin>>xc>>yc;

}
```

```

float kc(int xa, int ya, int xb, int yb)
{
    return sqrt((xb-xa)*(xb-xa)+(yb-ya)*(yb-ya));
}

float chuvi(int xa, int ya, int xb, int yb, int xc, int yc)
{
    float a=kc(xb,yb,xc,yc);
    float b=kc(xa,ya,xc,yc);
    float c=kc(xa,ya,xb,yb);
    return a+b+c;
}

float dientich(int xa, int ya, int xb, int yb, int xc, int yc)
{
    float a=kc(xb,yb,xc,yc);
    float b=kc(xa,ya,xc,yc);
    float c=kc(xa,ya,xb,yb);
    float p=chuvi(xa,ya,xb,yb,xc,yc)/2;
    float s=sqrt(p*(p-a)*(p-b)*(p-c));
    return s;
}

```

Thiết kế một số hàm cơ sở

(đây là những bài toán đã được giải ở chương 3; ở chương 4 này các bài toán được giải lại bằng cách sử dụng chương trình con)

Q1. Kiểm tra xem n có phải là số chính phương hay không ? (trả về 1/0)

Q2. Kiểm tra xem n có phải là số nguyên tố hay không ?

Q3. Kiểm tra xem n có phải là số hoàn chỉnh hay không ?

Q4. Hãy kiểm tra xem n có phải là số cân xứng hay không ? Ví dụ: 121, 10001, 44,...

Q5. Hãy tìm số đảo ngược của số n . Ví dụ: 2021->1202

Q6. Kiểm tra xem n có phải là số nguyên tố đối xứng hay không ?

Q7. Kiểm tra xem n có phải là số Armstrong hay không ?

Q8. Tìm UCLN của 2 số nguyên dương a, b .

//de bai xem tai slide 16/chuong 4.

```
#include<iostream.h>
#include<math.h>
void nhap(int &n);
int chinhphuong(int n);
int nguyento(int n);
int hoanchinh(int n);
int canxung(int n);
int sodaonguoc(int n);
int ntdoixung(int n);
int amrstrong(int n);
int main()
{
    int n;
    nhap(n);
    // cout<<chinhphuong(n);
    for (int i=1;i<=n;i++)
        if (amrstrong(i))
            cout<<i<<" ";
    cout<<endl;
    return 0;
}
```

```
void nhap(int &n)
{
    cout<<"Nhap so n = ";cin>>n;
}
//Q1
int chinhphuong(int n)
{
    return sqrt(n)==int(sqrt(n));
}
```

```
//Q2
//ver 1: dinh nghia SNT
int nguyento(int n)
{
    int d=0;
    for (int i=1;i<=n;i++)
        if (n%i==0) d++;
    return d==2;
}
//ver 2: tinh chat SNT
int nguyento(int n)
{
    if (n<2) return 0;
    int k=sqrt(n);
    for (int i=2;i<=k;i++)
        if (n%i==0) return 0;
    return 1;
}
```

//Q3

```
int hoanchinh(int n)
{
    int s=0;
    for (int i=1;i<=n/2;i++)
        if (n%i==0)
            s+=i;
    return s==n;
}
```

//phiên bản cải tiến

```
Int hoanchinh(int n) {
    if (n < 2) {
        return false;
    }
    int sum = 1; // 1 is a divisor of every number
    int sqrt_n = sqrt(n);
    for (int i = 2; i <= sqrt_n; i++)
        if (n % i == 0)
        {
            sum += i;
            if (i != n / i) sum += n / i;
        }
    return sum == n;
}
```

```
//Q4
int canxung(int n)
{
    int m=0,t=n;
    while (t>0)
    {
        m=m*10+t%10;
        t=t/10;
    }
    return m==n;
}
```

```
//Q5
int sodaonguoc(int n)
{
    int m=0;
    while (n>0)
    {
        m=m*10+n%10;
        n=n/10;
    }
    return m;
}
```

```
//Q6
int ntdoixung(int n)
{
    if (!nguyento(n)) return 0;
    int u=n+1;
    while (!nguyento(u)) u++;
    int v=n-1;
    while (!nguyento(v) && v>0) v--;
    return u+v==2*n;
}
```

```
//Q7
int amrstrong(int n)
{
    int d=0,t1=n;
    while (t1>0)
    {
        d++;
        t1=t1/10;
    }
    float s=0;int t2=n;
    while (t2>0)
    {
        s=s+pow(t2%10,d);
        t2=t2/10;
    }
    return s==n;
}
```

//Q8

```
#include<iostream.h>
void nhap(int &a, int &b);
int UCLN(int a, int b);
int BCNN(int a, int b);
int NTCN(int a, int b);
int main()
{
    int a,b;
    nhap(a,b);
    cout<<"UCLN : "<<UCLN(a,b)<<endl;
    cout<<"BCNN : "<<BCNN(a,b)<<endl;
    cout<<"NTCN : "<<NTCN(a,b)<<endl;
    cout<<endl;
    return 0;
}
```

```
void nhap(int &a, int &b)
{
    cout<<"Nhap 2 so a,b : ";cin>>a>>b;
}
int UCLN(int a, int b)
{
    int r=a%b;
    while (r!=0)
    {
        a=b;
        b=r;
        r=a%b;
    }
    return b;
}
```

```
int BCNN(int a, int b)
{
    return a*b/UCLN(a,b);
}
int NTCN(int a, int b)
{
    return UCLN(a,b)==1;
}
```

Ví dụ 1

Viết chương trình nhập vào hai phân số a/b và c/d (giả thiết a, b, c, d là các số nguyên không âm)

Hãy tính tổng, hiệu, tích, thương của hai phân số.

Yêu cầu phân số kết quả phải ở dạng tối giản

Ví dụ1: $1/2 + 1/6$ có kết quả là $2/3$.

Ví dụ 2: $1/2 + 3/4$ có kết quả là $5/4$.

Gợi ý thiết kế cho VD1

```
#include<iostream.h>
void nhapps(int &tu, int &mau);
void tongps(int tu1, int mau1, int tu2, int mau2,int &tu, int &mau);
void hieups(int tu1, int mau1, int tu2, int mau2,int &tu, int &mau);
void tichps(int tu1, int mau1, int tu2, int mau2,int &tu, int &mau);
void thuongps(int tu1, int mau1, int tu2, int mau2,int &tu, int &mau);
void xuatps(int tu, int mau);// toi gian phan so
int main()
{
    int tu1,mau1,tu2,mau2,tu,mau;
    nhapps(tu1,mau1);
    nhapps(tu2,mau2);
    tongps(tu1,mau1,tu2,mau2,tu,mau); xuatps(tu,mau);
    hieups(tu1,mau1,tu2,mau2,tu,mau); xuatps(tu,mau);
    tichps(tu1,mau1,tu2,mau2,tu,mau); xuatps(tu,mau);
    thuongps(tu1,mau1,tu2,mau2,tu,mau); xuatps(tu,mau);
    cout<<endl;
    return 0;
}
```

Ví dụ 2

VCT nhập vào số tự nhiên n .

a. Hãy đếm các số nguyên tố nhỏ hơn hoặc bằng n .

b. Hãy in các số nguyên tố nhỏ hơn hoặc bằng n .

c. Hãy in n số nguyên tố đầu tiên.

d. Hãy in các số nguyên tố nhỏ hơn hoặc bằng n mà khi đảo ngược số đó ta cũng được số nguyên tố.

e. số n được gọi là số nguyên tố đối xứng nếu n là số nguyên tố và bằng trung bình cộng của 2 số nguyên tố liền kề với nó.

Hãy in các số nguyên tố đối xứng nhỏ hơn hoặc bằng n .

f. Hãy in các số nguyên tố Mersenne nhỏ hơn hoặc bằng n .

Gợi ý thiết kế cho VD2

```
#include<iostream.h>
#include<math.h>
void nhap(int &n);
int caua(int n);
void caub(int n);
void cauc(int n);
void caud(int n);
void caue(int n);
void cauf(int n);
int main()
{
    int n;
    nhap(n);
    cout<<"Ket qua cau a : "<<caua(n)<<endl;
    caub(n);
    cout<<endl;
    return 0;
}
```

3.GIỚI THIỆU HÀM ĐỆ QUY

- Một hàm được gọi là có tính đệ quy (recursion) nếu trong bản thân hàm đó có lệnh gọi lại chính nó một cách trực tiếp hay gián tiếp.
- Một chương trình được gọi là có đệ quy nếu nó có chứa ít nhất một hàm đệ quy (gọi tắt là chương trình đệ quy).

(bài này giới thiệu về đệ quy cơ bản; nội dung chi tiết đầy đủ hơn về lập trình đệ quy SV sẽ nghiên cứu trong học phần Kỹ thuật lập trình)

Sơ đồ hàm đệ quy

Một hàm đệ quy (function) gồm 2 bước:

- **Bước cơ sở** (hay còn gọi là bước dừng):

Mô tả cấp độ giải được của bài toán.

- **Bước đệ quy:**

Bước gọi lại chính nó nhưng với cấp độ thấp hơn.

Hàm đệ qui hoạt động như thế nào ?

- Khi một hàm gọi đệ quy đến chính nó, chương trình sẽ tạo ra một tập các biến cục bộ hoàn toàn độc lập với các tập biến cục bộ đã được tạo ra trong các lần gọi trước đó.
- Có bao nhiêu lần gọi tới hàm đệ quy thì cũng có bấy nhiêu lần thoát ra khỏi hàm.
- Cứ mỗi lần thoát ra khỏi hàm thì một tập các biến cục bộ sẽ được giải phóng.
- Sự tương ứng giữa các lần gọi tới hàm và thoát ra khỏi hàm được thực hiện theo thứ tự ngược, nghĩa là lần ra đầu tiên ứng với lần vào cuối cùng và lần ra khỏi hàm cuối cùng ứng với lần đầu tiên gọi tới hàm (cơ chế vào sau ra trước).

Phân loại đệ quy

- Đệ quy tuyến tính,
 - Đệ quy nhị phân,
 - Đệ quy hồi tưởng,
 - Đệ quy phi tuyến
- ❖ (việc phân loại này chỉ mang tính hình thức)

Đệ quy tuyến tính

Một hàm được gọi là đệ quy tuyến tính (đệ quy đơn) nếu một lần gọi hàm nó chỉ phát sinh tối đa một lời gọi đệ quy.

Ví dụ 3.

Giai thừa được định nghĩa theo kiểu quy nạp như sau: $n! = n \cdot (n-1)!$
Viết chương trình tính $n!$, với n là một số nguyên không âm.

```
1. long giaithua(int n)
2. {
3.     if (n==0 || n==1)
4.         return 1;
5.     else
6.         return n*giaithua(n-1);
7. }
```

Đệ quy nhị phân

Một hàm được gọi là đệ quy nhị phân nếu mỗi lần gọi hàm nó phát sinh một số ít lời gọi đệ quy.

Ví dụ 4.

Viết chương trình tính số hạng thứ n của dãy fibonacci f_n xác định theo công thức quy nạp sau:

$$f_1=1,$$

$$f_2=1,$$

$$f_n = f_{n-1} + f_{n-2}, n \geq 3.$$

1. *int f(int n)*

2. {

3. *if (n==1 || n==2) return 1;*

4. *else return f(n-1)+f(n-2);*

}

Đệ quy hồi tương

Hai hàm P,Q được gọi là đệ quy hồi tương nếu hàm P có lời gọi đến hàm Q và ngược lại một cách trực tiếp hay gián tiếp.

Ví dụ 5.

Viết chương trình tính số hạng thứ n của hai dãy sau:

$$x_0 = 1, y_0 = 0,$$

$$x_n = x_{n-1} + y_{n-1} \text{ với mọi } n > 0,$$

$$y_n = 3 * x_{n-1} + 2 * y_{n-1} \text{ với mọi } n > 0.$$

1. *int tinhxn(int n)*

2. {

3. *if (n==0) return 1;*

4. *return tinhxn(n-1)+tinhyn(n-1);*

5. }

6. *int tinhyn(int n)*

7. {

8. *if (n==0) return 0;*

9. *return 3*tinhxn(n-1)+2*tinhyn(n-1);*

}

Đệ quy phi tuyến

- Một hàm được gọi là đệ quy phi tuyến (đệ quy phức) nếu mỗi lần gọi hàm thì nó phát sinh ra khoảng n lần gọi đệ quy.
- Thông thường với loại này, lời gọi đệ quy được đặt trong một hoặc nhiều vòng lặp.

Ví dụ 6.

Dãy A_n được cho như sau :

$$A_1=1$$

$$A_n=n(A_1+A_2+\dots+A_{n-1})$$

Viết hàm tính A_n có sử dụng đệ quy

```
1. int An(int n)
2. {
3.     if (n==1) return 1 ;
4.     int s=0 ;
5.     for (int i=1;i<n ;i++)
6.         s=s+An(i) ;
7.     return s*n ;
8. }
```

Khử đệ quy

- Không nên lạm dụng đệ quy và nếu một bài toán không quá khó để tìm được một lời giải không đệ quy thì nên chọn cách giải không đệ quy.
- Lời giải không đệ quy được gọi là khử đệ quy.
- SV dễ dàng khử đệ quy cho các bài toán tính $n!$, tính số hạng thứ n của dãy fibonacci

VD7 (đệ quy có nhớ)

```
1. #include<iostream.h>
2. long long fib(int n);
3. long long f[100];
4. int main()
5. {
6.     int n=50;
7.     f[1]=1;f[2]=1;
8.     cout<<fib(n);
9.     return 0;
10. }
```

```
11. long long fib(int n)
12. {
13.     if (!f[n])
14.         f[n]=fib(n-1)+fib(n-2);
15.     return f[n];
16. }
```